フォーマットファイル詳細検索モーダル UI仕様書

1. 概要

1.1 コンポーネントの目的

フォーマットファイル詳細検索モーダルは、MAIML統合テーブルを使用して、複数の検索条件を組み合わせてフォーマットファイル（XML/MAIML）を高速検索するためのモーダルダイアログです。最大5つまでの検索条件を組み合わせることができ、検索結果からファイルを選択できます。

1.2 使用場面

- **表示場所**: フォーマット管理画面のモーダルオーバーレイ
- **トリガー**:「詳細検索」ボタンのクリック
- **主な用途**:
  - 複数条件でのファイル検索
  - MAIML属性値による検索
  - 測定データによる検索
  - ファイル構造による検索

1.3 アクセス権限

- **必須ロール**: Experiment Admin（実験管理者）
- **認証**: 親コンポーネント（FormatManagement）で認証済み

---

2. 画面構成

2.1 モーダル構造


🔍 フォーマットファイル詳細検索 [×]
ℹ️ MAIML統合テーブル高速検索
高速化されたMaiML統合テーブル検索で、検索条件に一致する
フォーマットファイル（XML/MAIML）を表示します。
最大5つまで検索条件を組み合わせることができます。
検索条件 [条件追加] [リセット]
[1] [属性を選択 ▼] [検索値を入力]
AND
[2] [属性を選択 ▼] [検索値を入力] [×]
AND
[3] [属性を選択 ▼] [検索値を入力] [×]
[検索]
✅ ファイル数: 2件 | マッチ数: 3箇所 | 検索時間: 45ms
検索結果 [2件]
material_template_v1.xml [XML] [v1.0] [2マッチ]
formats/material/material_template_v1.xml
材料データのテンプレートファイル
[材料] [15.0 KB] [2024/11/20]
🎯 ヒット属性: [Document UUID: xxx] [ID属性: yyy]
measurement_protocol.maiml [MAIML] [v2.1] [1マッチ]
formats/measurement/measurement_protocol.maiml
測定プロトコルのMAIMLファイル
[測定] [22.5 KB] [2024/12/01]
🎯 ヒット属性: [Document UUID: xxx]

2.2 主要UI要素

2.2.1 モーダルヘッダー

- **アイコン**: SearchOutlined（赤色、#CF0027）
- **タイトル**: 「フォーマットファイル詳細検索」

2.2.2 検索説明エリア

- **タイプ**: Alert（info）
- **タイトル**: 「MAIML統合テーブル高速検索」
- **内容**: 機能説明と使用方法

2.2.3 検索条件カード

- **タイトル**: 「検索条件」
- **アクション**:
  - 条件追加ボタン（最大5件まで）
  - リセットボタン
- **検索条件行**:
  - 番号表示（赤背景）
  - 属性セレクトボックス
  - 検索値入力フィールド
  - 削除ボタン（2件目以降）
- **AND区切り**: 条件間にDivider
- **検索ボタン**: 大きなプライマリボタン

2.2.4 エラー表示エリア（条件付き）

- **タイプ**: Alert（error、closable）
- **表示条件**: エラー発生時のみ

2.2.5 検索統計カード（条件付き）

- **背景色**: 緑色（#E8F5E9）

- **表示項目**:
  - ファイル数
  - マッチ数
  - 検索時間
  - 検索条件数
- **表示条件**: 検索実行後

2.2.6 検索結果カード

- **タイトル**:「検索結果」
- **件数バッジ**: 検索結果がある場合のみ
- **結果ファイルカード**:
  - ファイル名とアイコン
  - ファイルタイプ・バージョン・マッチ数のTag
  - ファイルパス（コードスタイル）
  - 説明文
  - ファイル情報Tag（カテゴリ、サイズ、日付）
  - ヒット属性Tag

2.3 インタラクション

2.3.1 モーダル表示

- **操作**:「詳細検索」ボタンをクリック
- **結果**: モーダルが開き、検索条件が初期化される

2.3.2 検索条件の追加

- **操作**:「条件追加」ボタンをクリック
- **結果**: 新しい検索条件行が追加される（最大5件）

2.3.3 検索条件の削除

- **操作**: ×ボタンをクリック
- **結果**: 該当の検索条件行が削除される（最低1件は残る）

2.3.4 検索実行

- **操作**: 「検索」ボタンをクリック、またはEnterキーを押下
- **結果**:
  - 検索が実行される
  - ローディング状態が表示される
  - 検索結果と統計が表示される

2.3.5 リセット

- **操作**: 「リセット」ボタンをクリック
- **結果**:
  - 検索条件がクリアされる
  - 検索結果がクリアされる
  - エラーと統計がクリアされる

2.3.6 ファイル選択

- **操作**: 検索結果のファイルカードをクリック
- **結果**:
  - ファイルが選択される
  - `onFileSelect` コールバックが呼び出される
  - モーダルが閉じる

2.3.7 モーダルを閉じる

- **操作**: ×ボタン、またはモーダル外側をクリック、またはEscキーを押下
- **結果**:

- 全ての状態がクリアされる
- モーダルが閉じる

---

3. 機能仕様

3.1 検索可能な属性

3.1.1 属性カテゴリ

```
const ATTRIBUTE_CATEGORIES = {
  document: { label: 'Document情報', icon: '📄', color: '#CF0027' },
  data: { label: '測定データ', icon: '📊', color: '#0093CF' },
  structure: { label: 'ファイル構造', icon: '📐', color: '#ed6c02' },
  attributes: { label: 'MaiML属性', icon: '🏷️', color: '#2e7d32' },
};
```

カテゴリ説明:

カテゴリ	アイコン	色	説明
document	📄	#CF0027（赤）	Document要素の情報
data	📊	#0093CF（青）	測定データや値
structure	📐	#ed6c02（橙）	ファイルの構造情報
attributes	🏷️	#2e7d32（緑）	MAIML属性（NEW）

3.1.2 検索可能属性リスト

```
const SEARCHABLE_ATTRIBUTES = [
  // Document情報
  { value: 'document.uuid', label: 'Document UUID', category: 'document' },
  { value: 'document.name', label: 'Document名', category: 'document' },
  { value: 'document.description', label: 'Document説明', category: 'document' },

  // 測定データ
  { value: 'measurement_value', label: '測定値', category: 'data' },
```

```
  { value: 'units', label: '単位', category: 'data' },

  // ファイル構造
  { value: 'section_type', label: 'セクション種別', category: 'structure' },

  // MaiML属性（新規追加）
  { value: 'id', label: 'ID属性', category: 'attributes', isNew: true },
  { value: 'ref', label: 'Ref属性', category: 'attributes', isNew: true },
  { value: 'key', label: 'Key属性', category: 'attributes', isNew: true },
  { value: 'element', label: 'Element名', category: 'attributes', isNew: true },
  { value: 'value', label: '汎用Value', category: 'attributes', isNew: true },
];
```

NEW バッジ: `isNew: true` の属性には緑色の「NEW」バッジが表示される

3.1.3 UI動作

- ユーザー操作: 属性セレクトボックスをクリック
- 画面変化:
  - カテゴリ別にグループ化されたオプションが表示される
  - NEW属性には緑色のバッジが表示される
  - 検索（フィルター）が可能

3.2 検索条件の管理

3.2.1 検索条件の追加

```
const addSearchCriteria = () => {
  if (searchCriteria.length < 5) {
    setSearchCriteria([...searchCriteria, { attribute: '', value: '' }]);
  }
};
```

- **制限**: 最大5件まで
- **初期値**: 空の検索条件（`{ attribute: '', value: '' }`）

3.2.2 検索条件の更新

```
const updateSearchCriteria = (index: number, field: 'attribute' | 'value', value: string) => {
  const newCriteria = [...searchCriteria];
  newCriteria[index][field] = value;
  setSearchCriteria(newCriteria);
};
```

- **対象**: 指定インデックスの検索条件
- **更新可能フィールド**: `attribute`（属性）または `value`（検索値）

3.2.3 検索条件の削除

```
const removeSearchCriteria = (index: number) => {
  if (searchCriteria.length > 1) {
    const newCriteria = searchCriteria.filter((_, i) => i !== index);
    setSearchCriteria(newCriteria);
  }
};
```

- **制限**: 最低1件は残す
- **対象**: 指定インデックスの検索条件

3.2.4 検索条件のリセット

```
const handleReset = () => {
  setSearchCriteria([{ attribute: '', value: '' }]);
  setSearchResults([]);
  setError(null);
  setSearchStats(null);
};
```

- **リセット内容**:
  - 検索条件を初期状態（1件の空条件）に戻す

- 検索結果をクリア
- エラーをクリア
- 検索統計をクリア

3.2.5 UI動作

- ユーザー操作: 条件追加/削除/リセットボタンをクリック
- 画面変化:
  - 条件追加: 新しい検索条件行が表示される
  - 条件削除: 該当の検索条件行が削除される
  - リセット: 全ての状態がクリアされる

3.3 検索実行

3.3.1 バリデーション

```
const validCriteria = searchCriteria.filter(
  (criteria) => criteria.attribute && criteria.value.trim()
);

if (validCriteria.length === 0) {
  setError('検索条件を1つ以上設定してください');
  return;
}
```

- **検証内容**:
  - 属性が選択されている
  - 検索値が空でない（trimして空白のみでない）
- **エラー**: 有効な条件が1つもない場合

3.3.2 検索処理

```
const handleSearch = async () => {
  setError(null);
  setSearchStats(null);

  // バリデーション
  // ...

  const searchStartTime = Date.now();

  try {
    setIsSearching(true);
    console.log('🔍 フォーマットファイル検索開始:', validCriteria);

    // TODO: 実際のAPI呼び出しに置き換える
    // const results = await searchFormatFilesEnhanced(validCriteria);

    // モックデータ（開発用）
    const mockResults: SearchResult[] = [ /* ... */ ];

    const searchDuration = Date.now() - searchStartTime;

    setSearchResults(mockResults);
    setSearchStats({
      totalFiles: mockResults.length,
      totalMatches: mockResults.reduce((sum, file) => sum + file.matchCount, 0),
      searchDuration,
      totalCriteria: validCriteria.length,
    });

    if (mockResults.length === 0) {
      setError('検索条件に一致するファイルが見つかりませんでした');
    }
  } catch (err) {
    console.error('❌ 検索エラー:', err);
    const errorMessage = err instanceof Error ? err.message : '不明なエラーが発生しました';
    setError(`検索エラー: ${errorMessage}`);
    setSearchResults([]);
  } finally {
    setIsSearching(false);
  }
};
```

- **検索開始時**:
  - エラーと統計をクリア
  - ローディング状態を設定

- **検索中**:
  - API呼び出し（現在はモックデータ）
  - 検索時間を計測
- **検索完了時**:
  - 結果を設定
  - 統計を計算・設定
  - 結果が0件の場合、エラーメッセージを表示
- **エラー発生時**:
  - エラーメッセージを設定
  - 結果をクリア

3.3.3 検索統計の計算

```
setSearchStats({
  totalFiles: mockResults.length,                                          // ファイル数
  totalMatches: mockResults.reduce((sum, file) => sum + file.matchCount, 0), // マッチ数合計
  searchDuration,                                                          // 検索時間（ms）
  totalCriteria: validCriteria.length,                                     // 検索条件数
});
```

3.3.4 UI動作

- ユーザー操作: 「検索」ボタンをクリック、またはEnterキーを押下
- 画面変化:
  - 検索ボタンがローディング状態になる
  - 成功時: 検索結果と統計が表示される
  - 失敗時: エラーメッセージが表示される
  - 結果0件時: エラーメッセージと空状態が表示される

3.4 検索結果の表示

3.4.1 検索結果カードのレンダリング

各ファイルカードには以下の情報が表示されます：

1. **ファイル名とタイプ**
   - アイコン（FileTextOutlined、赤色）
   - ファイル名（太字）
   - ファイルタイプTag（XML=緑、MAIML=青）
   - バージョンTag（灰色）
   - マッチ数Tag（緑色）
2. **ファイルパス**
   - コードスタイル（灰色背景、等幅フォント）
   - フォルダアイコン付き
3. **ファイル説明**
   - 灰色テキスト
4. **ファイル情報Tag**
   - カテゴリTag（アイコン付き）
   - サイズTag（アイコン付き）
   - 日付Tag（アイコン付き）
5. **ヒット属性**
   - シアン色のTag
   - 属性名と値を表示

3.4.2 ファイルサイズのフォーマット

```
const formatFileSize = (bytes: number) => {
  if (bytes < 1024) return `${bytes} B`;
  if (bytes < 1024 * 1024) return `${(bytes / 1024).toFixed(1)} KB`;
```

```
  return `${(bytes / (1024 * 1024)).toFixed(1)} MB`;
};
```

3.4.3 カテゴリ表示名の取得

```
const getCategoryLabel = (category: string) => {
  const categoryMap: Record<string, string> = {
    protocol: 'プロトコル',
    material: '材料',
    measurement: '測定',
    analysis: '分析',
    sample: 'サンプル',
    device: 'デバイス',
  };
  return categoryMap[category] || category;
};
```

3.4.4 UI動作

- ユーザー操作: なし（自動表示）
- 画面変化:
  - 検索結果がカード形式で表示される
  - カードにマウスオーバーで視覚的フィードバック
  - カードクリックでファイルが選択される

3.5 ファイル選択

3.5.1 選択処理

```
const handleFileSelect = (file: SearchResult) => {
  console.log('📄 ファイル選択:', file.name);
  onFileSelect(file);
  handleClose();
};
```

- **動作**:
  1. コンソールにログ出力
  2. 親コンポーネントの `onFileSelect` コールバックを呼び出し
  3. モーダルを閉じる

3.5.2 UI動作

- ユーザー操作: ファイルカードをクリック
- 画面変化:
  - ファイルが選択される
  - モーダルが閉じる
  - 親コンポーネントで選択されたファイルが処理される

3.6 モーダルを閉じる

3.6.1 閉じる処理

```
const handleClose = () => {
  setSearchCriteria([{ attribute: '', value: '' }]);
  setSearchResults([]);
  setError(null);
  setSearchStats(null);
  onClose();
};
```

- **リセット内容**:
  - 検索条件を初期状態に戻す
  - 検索結果をクリア
  - エラーをクリア
  - 検索統計をクリア
  - 親コンポーネントの `onClose` コールバックを呼び出し

3.6.2 UI動作

- ユーザー操作: ×ボタン、またはモーダル外側をクリック、またはEscキーを押下
- 画面変化:
  - 全ての状態がクリアされる
  - モーダルが閉じる

---

4. データ仕様

4.1 Props定義

```
interface FormatSearchModalProps {
  open: boolean;                                    // モーダル表示状態
  onClose: () => void;                              // モーダルを閉じる時のコールバック
  onFileSelect: (file: SearchResult) => void;       // ファイル選択時のコールバック
}
```

4.2 SearchCriteria型定義

```
interface SearchCriteria {
  attribute: string;    // 検索属性（例: 'document.uuid', 'id'）
  value: string;        // 検索値
}
```

4.3 SearchResult型定義

```
interface SearchResult {
  id: string;                   // ファイルID
  name: string;                 // ファイル名
  path: string;                 // S3/Swiftパス
  size: number;                 // ファイルサイズ（バイト）
```

```
  lastModified: string;           // 最終更新日時（ISO 8601形式）
  type: string;                   // ファイルタイプ（'xml' | 'maiml'）
  version?: string;               // バージョン
  description?: string;           // 説明
  formatName: string;             // フォーマット名
  category: string;               // カテゴリ
  matchCount: number;             // マッチ数
  matchedAttributes?: Array<{     // ヒット属性リスト
    attribute: string;
    value: string;
  }>;
}
```

4.4 SearchStats型定義

```
interface SearchStats {
  totalFiles: number;       // 検索結果ファイル数
  totalMatches: number;     // マッチ箇所の合計数
  searchDuration: number;   // 検索時間（ミリ秒）
  totalCriteria: number;    // 使用した検索条件数
}
```

4.5 State定義

```
const [searchCriteria, setSearchCriteria] = useState<SearchCriteria[]>([
  { attribute: '', value: '' },
]);
const [searchResults, setSearchResults] = useState<SearchResult[]>([]);
const [isSearching, setIsSearching] = useState(false);
const [error, setError] = useState<string | null>(null);
const [searchStats, setSearchStats] = useState<SearchStats | null>(null);
```

---

5. API連携

5.1 想定されるAPI

現在はモックデータを使用していますが、将来的には以下のAPIを呼び出す予定：

```
// TODO: 実際のAPI呼び出しに置き換える
// const results = await searchFormatFilesEnhanced(validCriteria);
```

想定されるAPI:

- **POST** `/api/v1/experiment_admin/formats/search`
  - フォーマットファイル検索API
  - リクエストボディ: `{ criteria: SearchCriteria[] }`
  - レスポンス: `SearchResult[]`

---

6. エラーハンドリング

6.1 エラーケースと表示方法

エラーケース	検知タイミング	表示方法	表示内容
検索条件未設定	検索実行時	Alert（error）	「検索条件を1つ以上設定してください」
検索結果0件	API返却時	Alert（error）	「検索条件に一致するファイルが見つかりませんでした」
API通信エラー	API返却時	Alert（error）	「検索エラー: {エラーメッセージ}」

---

7. 実装ファイル

7.1 ファイルパス

- **メインファイル**: `frontend/src/components/project/FormatSearchModal.tsx`

7.2 関連ファイル

- **親コンポーネント**: `frontend/src/components/project/FormatManagement.tsx`
- **コンテキスト**: `frontend/src/contexts/LanguageContext.tsx`

7.3 依存ライブラリ

- **Ant Design**: Modal, Input, Select, Button, Space, Tag, Typography, Divider, Alert, Card, Empty
- **Ant Design Icons**: SearchOutlined, PlusOutlined, CloseOutlined, ReloadOutlined, FileTextOutlined

---

8. UI/UXの考慮事項

8.1 レスポンシブデザイン

- **幅**: 900px（固定）
- **高さ**: 最大calc(100vh - 200px)、スクロール可能

8.2 アクセシビリティ

- **キーボード操作**: Enterキーで検索、Escキーでモーダルを閉じる
- **視覚的フィードバック**: ホバー時のカード変化

8.3 パフォーマンス

- **useMemo**: 属性オプションと検索条件の有効性チェックをメモ化

---

9. 変更履歴

日付	変更内容	担当者
2026-03-11	実装ベースで仕様書を作成（Phase 4: フォーマット管理画面）	Cline